Agentic架构 1：反思（Reflection）

> **反思(Reflection)** —— 这种模式将大语言模型从一个简单的、单次生成器提升为一个**更具思辨性和稳健性的推理者**。反思型Agent不会仅仅给出它想到的第一个答案，而是会退一步，对自己的工作进行**批判、分析和完善**。这种**自我完善的迭代过程**，是构建更可靠、更高质量AI系统的基石。

定义

反思架构涉及一个Agent在对自身输出进行**批判和修改**后再返回最终答案。它并非进行单次生成，而是进行一个**多步骤的内部独白：生成、评估、改进**。这模仿了人类起草、审阅和编辑以发现错误、提升质量的过程。

高层工作流

1. **生成**：Agent根据用户的提示词生成一个**初步的草稿或解决方案**。
2. **批判**：随后，Agent转换角色，成为一个**评论家**。它会向自己提问，例如："这个答案可能存在什么问题？"、"遗漏了什么？"、"这个解决方案是最优的吗？"、"是否存在任何逻辑缺陷或错误？"
3. **完善**：利用自我批判中获得的洞察，Agent生成最终**改进后**的输出版本。

适用场景 / 应用

- **代码生成**：初始代码可能存在错误、效率低下或缺少注释。反思使Agent能够充当自己的**代码审查员**，在呈现最终脚本之前**捕获错误并改进代码风格**。
- **复杂内容总结**：在总结内容密集的文档时，首次尝试可能会遗漏细微差别或关键细节。反思步骤有助于确保**总结全面且准确**。
- **创意写作与内容创作**：电子邮件、博客文章或故事的初稿总有改进空间。反思使Agent能够**优化其语气、清晰度和影响力**。

优势与劣势

- **优势**：
  - **质量提升**：**直接定位并修正错误**，从而产生更准确、更稳健、更严谨的输出。
  - **低开销**：这是一个概念上简单的模式，可以使用单个LLM实现，**无需复杂的外部工具**。
- **劣势**：
  - **自我偏见**：Agent仍然受限于其**自身的知识和偏见**。如果它不知道解决问题的更好方法，就无法通过批判来获得更好的解决方案。它可以修正它能够识别出的缺陷，但**无法创造它所缺乏的知识**。
  - **增加延迟与成本**：该过程至少涉及**两次LLM调用（生成 + 批判/完善）**，因此比单次生成方法更慢、成本更高。

阶段 0：基础与环境搭建

在构建**反思型Agent**之前，需要先搭建好环境。包括安装必要的库、导入模块以及配置API密钥。

步骤 0.1：安装核心库

安装本项目所需的Python核心库。

- `langchain-nebius` 包用于访问 Nebius AI Studio 模型
- `langchain` 和 `langgraph` 将提供核心编排框架
- `python-dotenv` 用于管理API密钥
- `rich` 便于以美观的格式打印输出

```
pip install -q -U langchain-nebius langchain langgraph rich python-dotenv
```

步骤 0.2：导入库与配置密钥

从已安装的库中导入所有必要的组件。

- 使用 **`python-dotenv` 库**，从本地的 `.env` 文件中安全地加载 Nebius API 密钥。
- 设置 **LangSmith** 用于追踪，这对于调试多步骤的 Agentic 工作流来说非常有价值。
- **需要执行的操作：** 在目录中创建一个名为 `.env` 的文件，并将密钥添加到其中，格式如下：

  ```
  NEBIUS_API_KEY="your_nebius_api_key_here"
  LANGCHAIN_API_KEY="your_langsmith_api_key_here"
  ```

```
import os
import json
from typing import List, TypedDict, Optional
from dotenv import load_dotenv

# Nebius 和 LangChain 组件
from langchain_nebius import ChatNebius
from pydantic import BaseModel, Field  # 修正为 Pydantic v2 的导入路径
from langgraph.graph import StateGraph, END

# 美化打印输出
from rich.console import Console
from rich.markdown import Markdown
from rich.syntax import Syntax

# --- API 密钥与追踪配置 ---
load_dotenv()  # 加载 .env 文件中的环境变量

# 配置 LangSmith 追踪功能
os.environ["LANGCHAIN_TRACING_V2"] = "true"  # 启用 LangSmith 追踪 v2 版本
os.environ["LANGCHAIN_PROJECT"] = "Agentic Architecture - Reflection
(Nebius)"  # 设置追踪项目名称

# 校验必需的环境变量是否已配置
if not os.environ.get("NEBIUS_API_KEY"):
    print("NEBIUS_API_KEY not found. Please create a .env file and set
it.")  # Nebius API 密钥缺失提示
```

```
if not os.environ.get("LANGCHAIN_API_KEY"):
    print("LANGCHAIN_API_KEY not found. Please create a .env file and set
it for tracing.")  # LangChain API 密钥缺失提示

print("Environment variables loaded and tracing is set up.")  # 环境变量加载
完成提示
```

阶段 1：构建反思的核心组件

一个健壮的反思架构不仅仅是简单的提示词，而是需要构建为一个**结构化的三部分系统：生成器、评论家和完善器**。为了确保可靠性，使用 **Pydantic 模型**来为每个步骤定义期望的输出模式。

步骤 1.1：使用 Pydantic 定义数据模式

定义 Pydantic 模型，作为我们与 LLM 之间的约定。这告诉 LLM 它的输出应该具有确切的结构，这对于多步骤流程至关重要，因为一个步骤的输出将成为下一个步骤的输入。

```
class DraftCode(BaseModel):
    """用于存储智能体生成的初始代码草稿的结构。"""
    code: str = Field(description="为解决用户需求而生成的 Python 代码。")
    explanation: str = Field(description="代码工作原理的简要说明。")

class Critique(BaseModel):
    """用于存储对生成代码的自我批判结果的结构。"""
    has_errors: bool = Field(description="代码是否存在潜在的缺陷或逻辑错误？")
    is_efficient: bool = Field(description="代码是否以高效且优化的方式编写？")
    suggested_improvements: List[str] = Field(description="针对代码改进的具
体、可操作的建议。")
    critique_summary: str = Field(description="批判内容的总结。")

class RefinedCode(BaseModel):
    """用于存储采纳批判后最终精炼代码的结构。"""
    refined_code: str = Field(description="最终改进后的 Python 代码。")
    refinement_summary: str = Field(description="基于批判所做修改的总结。")

print("Pydantic models for Draft, Critique, and RefinedCode have been
defined.")  # 提示 Pydantic 模型已定义完成
```

> **关于输出的讨论：** 成功定义了数据结构。`Critique` **模型尤其重要**；通过要求 `has_errors` 和 `is_efficient` 等特定字段，我们引导 LLM 进行比单纯要求"审查代码"**更加结构化、更有用的评估**。

步骤 1.2：初始化 Nebius LLM 和控制台

初始化 Nebius 语言模型，它将驱动所有三个角色（**生成器、评论家和完善器**）。

- 使用像 `meta-llama/Meta-Llama-3.1-8B-Instruct` 这样强大的**模型**，以确保所有步骤都能进行高质量的推理。
- 设置 `rich` 控制台，以实现**清晰、格式化的输出**。

```
# 使用 Nebius 提供的强大模型进行代码生成与自我批判
llm = ChatNebius(
    model="meta-llama/Meta-Llama-3.1-8B-Instruct",  # 指定使用的 Llama 3.1
8B 指令微调模型
    temperature=0.2  # 设置较低的温度参数以控制输出随机性，确保生成结果更稳定
)

# 初始化 Rich 控制台，用于美化打印输出
console = Console()
# 提示 Nebius 大语言模型和控制台已初始化完成
print("Nebius LLM and Console are initialized.")
```

步骤 1.3：创建生成器节点

生成器节点的唯一任务就是**接收用户的请求并生成初稿**。把 `DraftCode` **Pydantic 模型**绑定到 Nebius LLM，以确保其输出具有正确的结构。

```
def generator_node(state):
    """生成代码的初始草稿。"""
    console.print("--- 1. Generating Initial Draft ---")  # 在控制台输出当前阶
段提示

    # 将基础 LLM 包装为结构化输出模型，确保输出符合 DraftCode 的结构定义
    generator_llm = llm.with_structured_output(DraftCode)

    # 构建提示词，要求模型扮演 Python 专家角色生成代码
    prompt = f"""You are an expert Python programmer. Write a Python
function to solve the following request.
    Provide a simple, clear implementation and an explanation.

    Request: {state['user_request']}
    """

    # 调用 LLM 生成代码草稿
    draft = generator_llm.invoke(prompt)

    # 返回生成结果，使用 model_dump() 将 Pydantic 模型序列化为字典格式以便在状态中传
递
    return {"draft": draft.model_dump()}  # 修正：使用 model_dump() 替代已弃用
的 dict()
```

步骤 1.4：创建评论家节点

评论家节点是反思过程的**核心**。该节点接收初稿，**分析其中的缺陷**，并使用我们的 `Critique` Pydantic 模型生成**结构化的批判意见**。

```
def critic_node(state):
    """对生成的代码进行批判性评估，识别错误与效率问题。"""
```

```
    console.print("--- 2. Critiquing Draft ---")  # 在控制台输出当前阶段提示

    # 将基础 LLM 包装为结构化输出模型，确保输出符合 Critique 的结构定义
    critic_llm = llm.with_structured_output(Critique)

    # 从状态中提取待批判的代码
    code_to_critique = state['draft']['code']

    # 构建批判提示词，要求模型扮演资深代码审查者角色
    prompt = f"""You are an expert code reviewer and senior Python
developer. Your task is to perform a thorough critique of the following
code.

    Analyze the code for:
    1.  **Bugs and Errors:** Are there any potential runtime errors,
logical flaws, or edge cases that are not handled?
    2.  **Efficiency and Best Practices:** Is this the most efficient way
to solve the problem? Does it follow standard Python conventions (PEP 8)?

    Provide a structured critique with specific, actionable suggestions.

    Code to Review:
    \`\`\`python
    {code_to_critique}
    \`\`\`
    """

    # 调用 LLM 生成批判结果
    critique = critic_llm.invoke(prompt)

    # 返回批判结果，使用 model_dump() 将 Pydantic 模型序列化为字典格式以便在状态中传
递
    return {"critique": critique.model_dump()}  # 修正：使用 model_dump() 替
代已弃用的 dict()
```

步骤 1.5：创建完善器节点

完善器节点接收原始草稿和结构化的批判意见，其任务是**编写最终改进后的代码版本**。

```
def refiner_node(state):
    """根据批判结果对代码进行精炼改进。"""
    console.print("--- 3. Refining Code ---")  # 在控制台输出当前阶段提示

    # 将基础 LLM 包装为结构化输出模型，确保输出符合 RefinedCode 的结构定义
    refiner_llm = llm.with_structured_output(RefinedCode)

    # 从状态中提取原始草稿代码
    draft_code = state['draft']['code']

    # 将批判结果格式化为 JSON 字符串，便于在提示词中呈现（增加缩进提升可读性）
    critique_suggestions = json.dumps(state['critique'], indent=2)
```

```
    # 构建精炼提示词，要求模型根据批判建议重写代码
    prompt = f"""You are an expert Python programmer tasked with refining a
piece of code based on a critique.

    Your goal is to rewrite the original code, implementing all the
suggested improvements from the critique.

    **Original Code:**
    \`\`\`python
    {draft_code}
    \`\`\`

    **Critique and Suggestions:**
    {critique_suggestions}

    Please provide the final, refined code and a summary of the changes you
made.
    """

    # 调用 LLM 生成精炼后的代码
    refined_code = refiner_llm.invoke(prompt)

    # 返回精炼结果，使用 model_dump() 将 Pydantic 模型序列化为字典格式以便在状态中传
递
    return {"refined_code": refined_code.model_dump()}  # 修正：使用
model_dump() 替代已弃用的 dict()
```

> **阶段 1 讨论：** 本阶段创建了**反思型Agent的三个核心逻辑组件**。每个组件都是一个**独立的函数（或称"节点"）**，执行一个**单一、定义明确的任务**。在每个阶段使用**结构化输出**，确保了数据能够可靠地从一个节点流向下一个节点。后续则准备使用 **LangGraph** 来编排这个工作流。

阶段 2：使用 LangGraph 编排反思工作流

步骤 2.1：定义图状态

"状态"是图结构中的记忆。它是一个**中心对象**，在节点之间传递，每个节点都可以从中**读取**数据或向其中**写入**数据。将使用 Python 的 `TypedDict` 定义一个 `ReflectionState`，用于**存放工作流中的所有数据片段**。

```
class ReflectionState(TypedDict):
    """表示反思工作流图的状态结构。"""
    user_request: str           # 用户原始请求
    draft: Optional[dict]       # 初始代码草稿（DraftCode 模型的字典形式）
    critique: Optional[dict]    # 批判结果（Critique 模型的字典形式）
    refined_code: Optional[dict]  # 精炼后的代码（RefinedCode 模型的字典形式）

print("ReflectionState TypedDict defined.")  # 提示状态类型已定义完成
```

步骤 2.2：构建并可视化图结构

使用 `StateGraph` 把节点组装成一个连贯的**工作流**。对于这个反思模式，工作流是一个简单的线性序列：**生成 → 批判 → 完善**。首先定义这个流程，然后**编译并可视化该图结构**，以确认其结构正确。

```
# 创建状态图构建器，指定使用 ReflectionState 作为状态结构
graph_builder = StateGraph(ReflectionState)

# 向图中添加节点，每个节点对应一个处理函数
graph_builder.add_node("generator", generator_node)  # 生成节点：负责生成初始代码草稿
graph_builder.add_node("critic", critic_node)        # 批判节点：负责对代码进行批判性评估
graph_builder.add_node("refiner", refiner_node)      # 精炼节点：负责根据批判结果改进代码

# 定义工作流的边（执行顺序）
graph_builder.set_entry_point("generator")  # 设置入口节点为 generator
graph_builder.add_edge("generator", "critic")  # generator 执行后进入 critic
graph_builder.add_edge("critic", "refiner")    # critic 执行后进入 refiner
graph_builder.add_edge("refiner", END)         # refiner 执行后结束流程

# 编译图，生成可执行的应用实例
reflection_app = graph_builder.compile()

print("Reflection graph compiled successfully!")  # 提示图编译成功

# 可视化图结构（用于调试和展示）
try:
    from IPython.display import Image, display  # 导入 IPython 可视化相关模块
    png_image = reflection_app.get_graph().draw_png()  # 获取图的 PNG 图像数据
    display(Image(png_image))  # 在 Jupyter 环境中显示图像
except Exception as e:
    # 如果可视化失败（如缺少 pygraphviz 依赖），打印错误信息，不影响主流程运行
    print(f"Graph visualization failed: {e}. Please ensure pygraphviz is installed.")
```

> **关于输出的讨论：** 图结构已成功编译。可视化结果确认预期的线性工作流。由此可见，**状态从入口点（`generator`）流入，经过 `critic` 和 `refiner` 节点，最终到达 `__end__` 状态**。这个简单而强大的结构现已准备好执行。

阶段 3：端到端执行与评估

图结构已编译完成，现在该看看反思模式的实际效果了。考虑给它一个编码任务，这个任务中，初次尝试可能不够理想，因此非常适合作为**自我批判和完善**的测试用例。

步骤 3.1：运行完整的反思工作流

调用已编译的 LangGraph 应用，请求编写一个用于计算第 n 个斐波那契数的函数。流式传输结果，并妥善累积完整的状态，以便最后能够检查所有中间步骤。

```
# 定义用户请求：编写一个计算第 n 个斐波那契数的 Python 函数
user_request = "Write a Python function to find the nth Fibonacci number."

# 构建初始状态输入，仅包含用户请求字段（其他字段会在流程中逐步填充）
initial_input = {"user_request": user_request}

# 在控制台以青色粗体显示流程启动信息
console.print(f"[bold cyan]🚀 Kicking off Reflection workflow for request:
[/bold cyan] '{user_request}'\n")

# 流式执行反思工作流，逐个接收状态更新
# 修正：此循环正确捕获最终完整填充的状态
final_state = None
for state_update in reflection_app.stream(initial_input,
stream_mode="values"):
    # 每次迭代接收到当前步骤的状态快照，循环结束后 final_state 为最后一步的状态
    final_state = state_update

# 在控制台以绿色粗体显示流程完成信息
console.print("\n[bold green]✅ Reflection workflow complete![/bold
green]")
```

步骤 3.2：分析"前后对比"

检查工作流每个阶段的输出，这些输出都存储在 `final_state` 中。通过打印初稿、收到的批判意见以及最终完善后的代码，以清晰地看到**反思**过程所带来的价值。

```
# 校验最终状态是否存在且包含所有必需的字段（草稿、批判、精炼代码）
if final_state and 'draft' in final_state and 'critique' in final_state and
'refined_code' in final_state:
    # 以 Markdown 格式输出初始草稿部分标题
    console.print(Markdown("--- ### Initial Draft ---"))

    # 输出草稿代码的解释说明
    console.print(Markdown(f"**Explanation:** {final_state['draft']
['explanation']}"))

    # 使用 Rich 的 Syntax 组件进行代码高亮显示（主题：monokai，显示行号）
    console.print(Syntax(final_state['draft']['code'], "python",
theme="monokai", line_numbers=True))

    # 以 Markdown 格式输出批判部分标题
    console.print(Markdown("\n--- ### Critique ---"))

    # 输出批判总结
    console.print(Markdown(f"**Summary:** {final_state['critique']
['critique_summary']}"))

    # 输出改进建议列表
    console.print(Markdown(f"**Improvements Suggested:**"))
```

```
    for improvement in final_state['critique']['suggested_improvements']:
        # 逐条输出改进建议，使用 Markdown 列表格式
        console.print(Markdown(f"- {improvement}"))

    # 以 Markdown 格式输出最终精炼代码部分标题
    console.print(Markdown("\n--- ### Final Refined Code ---"))

    # 输出精炼过程的修改总结
    console.print(Markdown(f"**Refinement Summary:**
{final_state['refined_code']['refinement_summary']}"))

    # 使用 Rich 的 Syntax 组件对精炼后的代码进行高亮显示（主题：monokai，显示行号）
    console.print(Syntax(final_state['refined_code']['refined_code'],
"python", theme="monokai", line_numbers=True))
else:
    # 若状态数据不完整，输出错误提示信息（红色粗体）
    console.print("[bold red]Error: The `final_state` is not available or
is incomplete. Please check the execution of the previous cells.[/bold
red]")
```

> **关于输出的讨论：** 这个结果完美地展示了**反思模式**的力量。
>
> 1. **初始草稿**很可能生成了一个简单的递归解决方案。虽然正确，但由于重复计算相同的值，这种方法**效率极低**，导致指数级时间复杂度。
> 2. **批判意见**正确地识别出了这个主要缺陷。LLM 扮演 **"评论家"角色**时，指出了效率低下的问题，并建议采用**更优**的迭代方法来避免冗余计算。
> 3. **最终完善后的代码**成功地采纳了**批判意见**。它将缓慢的递归函数替换为一个更快的迭代解决方案，使用循环和两个变量来追踪数列。
>
> 这是一个不容小觑的改进。Agent 不仅仅是修正了一个拼写错误；**它从根本上改变了算法，以得到一个更健壮、更具扩展性的解决方案**。这正是反思模式的价值所在。

步骤 3.3：定量评估（LLM 作为评判者）

为了使分析更加规范化，**使用另一个 LLM 作为公正的"评判者"**，对初稿和最终代码的质量进行评分。这为衡量通过**反思**所获得的改进提供了一个**更加客观的度量标准**。

```
class CodeEvaluation(BaseModel):
    """用于评估代码质量的结构。"""
    correctness_score: int = Field(description="代码逻辑正确性评分，1-10分。")
    efficiency_score: int = Field(description="算法效率评分，1-10分。")
    style_score: int = Field(description="代码风格与可读性评分（遵循 PEP 8 标
准），1-10分。")
    justification: str = Field(description="各项评分的简要说明。")

# 将基础 LLM 包装为结构化输出模型，确保输出符合 CodeEvaluation 的结构定义
judge_llm = llm.with_structured_output(CodeEvaluation)

def evaluate_code(code_to_evaluate: str):
    """对给定的 Python 代码进行质量评估，返回评分和说明。"""
    # 构建评估提示词，要求模型扮演专家评审角色
```

```
    prompt = f"""You are an expert judge of Python code. Evaluate the
following function on a scale of 1-10 for correctness, efficiency, and
style. Provide a brief justification.

    Code:
    \`\`\`python
    {code_to_evaluate}
    \`\`\`
    """
    # 调用 LLM 生成评估结果
    return judge_llm.invoke(prompt)

# 校验最终状态是否包含评估所需的字段（草稿和精炼代码）
if final_state and 'draft' in final_state and 'refined_code' in
final_state:
    # 评估初始草稿代码
    console.print("--- Evaluating Initial Draft ---")
    initial_draft_evaluation = evaluate_code(final_state['draft']['code'])
    console.print(initial_draft_evaluation.model_dump())  # 修正：使用
model_dump() 替代已弃用的 dict()

    # 评估精炼后的代码
    console.print("\n--- Evaluating Refined Code ---")
    refined_code_evaluation = evaluate_code(final_state['refined_code']
['refined_code'])
    console.print(refined_code_evaluation.model_dump())  # 修正：使用
model_dump() 替代已弃用的 dict()
else:
    # 若状态数据不完整，输出错误提示信息（红色粗体）
    console.print("[bold red]Error: Cannot perform evaluation because the
`final_state` is incomplete.[/bold red]")
```

> **关于输出的讨论： LLM 作为评判者**的评估为反思模式的成功提供了定量证据。初稿可能在正确性方面获得了高分，但效率方面得分很低。相比之下，完善后的代码在正确性和效率方面都会获得高分。这种自动化的评分评估证实了反思过程不仅仅是改变了代码——**它以一种可衡量的方式切实地改进了代码**。

结论

在本项目中，我们使用 **Nebius AI Studio 模型**，基于**反思**架构成功**构建**、**执行并评估**了一个完整的端到端 Agent。我们亲眼见证了这种简单而强大的模式如何将一个基础的 LLM 生成器转变为一个**更复杂**、**更可靠的问题解决者**。

通过将流程结构化为**生成**、**批判**和**完善**三个不同的步骤，并使用 **LangGraph** 进行编排，我们创建了一个能够**识别并修正**自身重大缺陷的健壮系统。从低效的递归解决方案到最优的迭代解决方案这一切实的改进，证明了**反思是超越简单 Agent 任务、构建展现出更深层次质量和审慎能力的 AI 系统的基础技术**。